



# Explainable Network Intrusion Detection and Response

A system that detects SYN floods and port scans with transparent rules, and records the evidence, risk score, decision and response behind every alert.

## Project report

|              |                 |
|--------------|-----------------|
| Submitted by | [Student names] |
|--------------|-----------------|

|                  |                    |
|------------------|--------------------|
| Register numbers | [Register numbers] |
|------------------|--------------------|

|            |              |
|------------|--------------|
| Department | [Department] |
|------------|--------------|

|             |                         |
|-------------|-------------------------|
| Institution | [College or university] |
|-------------|-------------------------|

|               |                              |
|---------------|------------------------------|
| Project guide | [Guide name and designation] |
|---------------|------------------------------|

|               |        |
|---------------|--------|
| Academic year | [Year] |
|---------------|--------|

|             |                                                                         |
|-------------|-------------------------------------------------------------------------|
| Source code | <a href="https://github.com/RR-1902/aegis">github.com/RR-1902/aegis</a> |
|-------------|-------------------------------------------------------------------------|

# Abstract

AEGIS is a network intrusion detection and response system built so that every alert can be checked by a person. It captures packets, groups them into flows measured over five-second windows, and tests each window against two explicit rules: one for SYN floods and one for port scans.

Each rule that fires adds fixed points to a risk score from 0 to 100. A conservative policy engine turns the resulting risk level into a recommended action, and a simulation-only response engine validates it without ever touching the host or its firewall. The whole chain is stored as one immutable security event, identified by a SHA-256 hash of what was observed, and served through a REST API.

A web interface explains the system step by step. It follows one real attack through all ten stages, lets the reader try the rules, runs attack simulations through the real pipeline, and shows the full decision chain of every stored event.

|                                               |                                                |                                                   |                                                      |                                             |
|-----------------------------------------------|------------------------------------------------|---------------------------------------------------|------------------------------------------------------|---------------------------------------------|
| <b>10</b><br>pipeline stages, one module each | <b>28</b><br>features measured per flow window | <b>2</b><br>detection rules with fixed thresholds | <b>310</b><br>automated tests, backend and interface | <b>0</b><br>firewall commands ever executed |
|-----------------------------------------------|------------------------------------------------|---------------------------------------------------|------------------------------------------------------|---------------------------------------------|

## 1 Introduction

Most detectors report a verdict without its reasons. AEGIS is built to report both.

Intrusion detection systems watch network traffic and raise alerts. Many modern systems rely on statistical or machine-learning models whose outputs are hard to explain: an analyst sees a score, not the measurement behind it. For teaching, auditing and incident review, a reviewer needs to know which value crossed which limit and what the system did next.

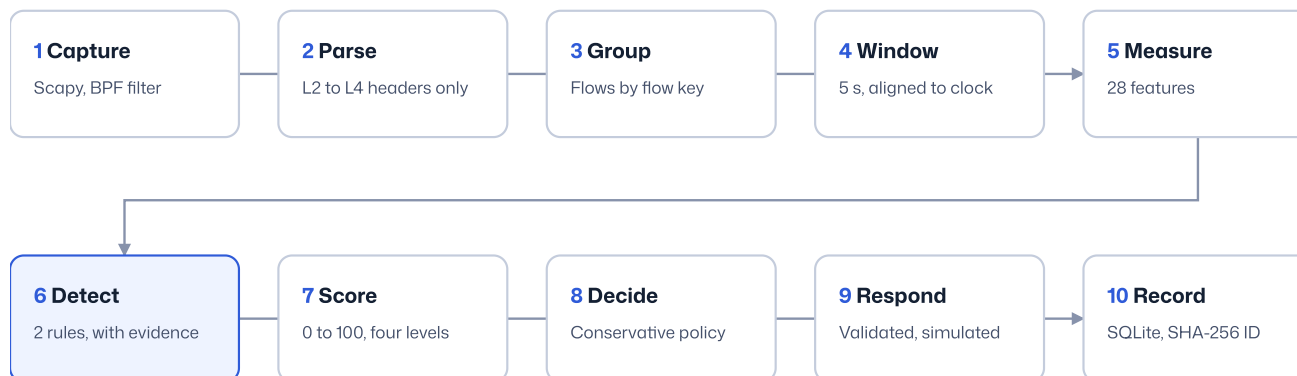
### Objectives

1. Capture and parse live traffic at the Ethernet, network and transport layers.
2. Detect SYN floods and port scans with explicit, configurable thresholds, keeping the evidence.
3. Turn detections into a transparent risk score and a proportionate, safe response.
4. Store each decision chain as an immutable, verifiable record and serve it over an API.
5. Explain the whole system, and every stored event, in a web interface.

The name comes from Greek myth, where the aegis was the shield of Zeus and Athena. To act under someone's aegis is to act under their protection.

## 2 How AEGIS works

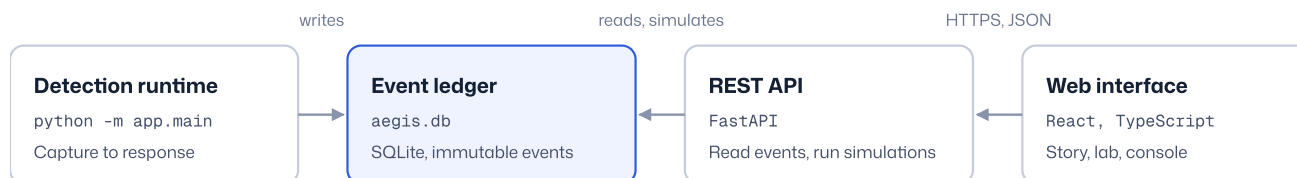
Packets pass through ten small stages. Each stage has one job, and each records why it did what it did.



**Figure 1.** The detection pipeline. A window in which no rule fires stops at stage 6 and nothing is stored.

### Components

The detection runtime and the API run as separate processes and share only the SQLite file, so nothing on the web side can change what the engine recorded. The one write the API offers, an attack simulation, still passes through the full pipeline.



**Figure 2.** Processes and data flow.

### The security event

Each alert is stored once as a security event: the flow, the window, every detection with its evidence, the risk score, the policy decision and the response. Its ID is security-event : followed by the SHA-256 hash of the flow key and window, so the same traffic always produces the same ID. An identical event saved twice is ignored, and a different event under an existing ID is refused.

### 3 Detection, scoring and response

Two rules, fixed points and a conservative policy. Every value below is configuration, not learned.

#### Detection rules

| Rule             | Fires when, in one window                                                                 | Medium severity                                   | High severity                 |
|------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------|-------------------------------|
| <b>SYN Flood</b> | $\text{syn\_rate} \geq 10$ per second and $\text{incomplete\_connection\_ratio} \geq 0.7$ | A value sits exactly on its threshold: <b>+35</b> | Both values above: <b>+55</b> |
| <b>Port Scan</b> | $\text{unique\_destination\_ports} \geq 20$                                               | Exactly 20 ports: <b>+25</b>                      | More than 20: <b>+40</b>      |

Each detection keeps its evidence: the feature values used, the thresholds applied and every comparison with its result, plus a one-sentence explanation.

#### From score to action

| Risk level      | Score  | Policy decision                                                            | Response                                                                           |
|-----------------|--------|----------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Low</b>      | 0-29   | Log only                                                                   | No action                                                                          |
| <b>Medium</b>   | 30-59  | Alert only                                                                 | No action                                                                          |
| <b>High</b>     | 60-79  | Block the source, only if the evidence names an observed source; otherwise | Simulated while SAFE_MODE is on; rejected if it is off, because no executor exists |
| <b>Critical</b> | 80-100 | alert only                                                                 |                                                                                    |

Points from every rule that fired on the same window are added and capped at 100. The policy never infers an attacker from a flow key, because a flow key can be spoofed. The response engine runs six validation checks before it records any outcome, and no code path runs a system or firewall command.

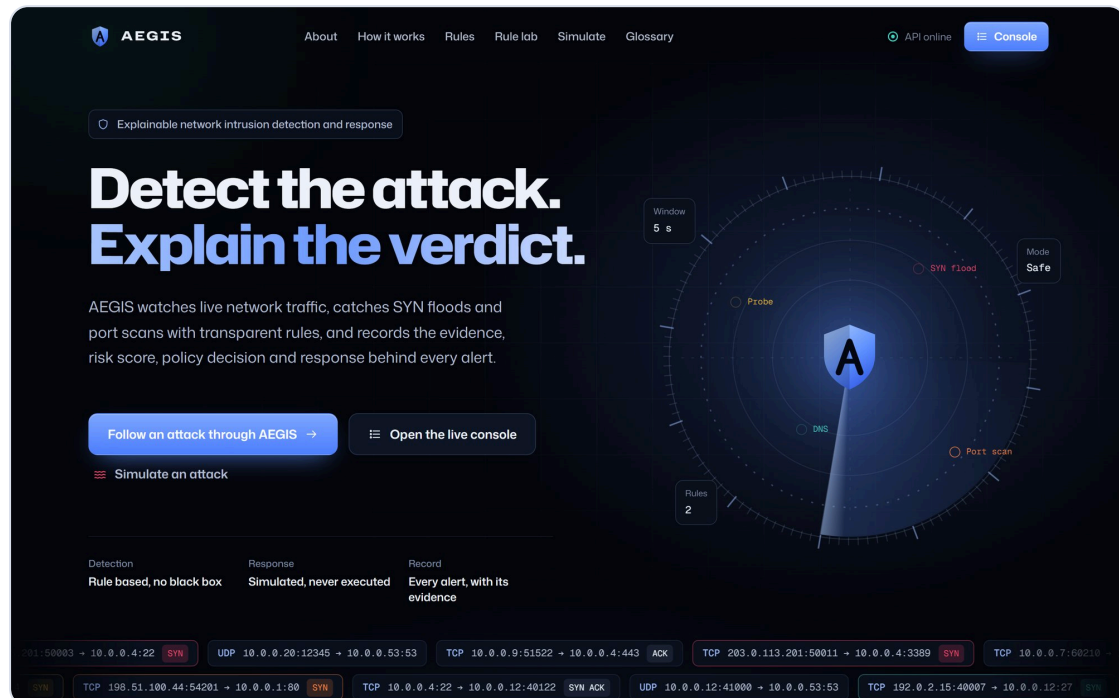
## 4 Implementation

| Layer           | Technology                                | Role                                                                           |
|-----------------|-------------------------------------------|--------------------------------------------------------------------------------|
| Engine          | Python 3.11, Scapy 2.6, Pydantic Settings | Capture, parsing, flows, features, rules, scoring, policy, response            |
| Storage and API | SQLite, FastAPI                           | Immutable event ledger; /health, /events, /simulations                         |
| Simulation      | In-memory Scapy packets                   | Five attack scenarios run through the real pipeline; nothing sent on a network |
| Web interface   | React 18, TypeScript, Vite, GSAP, Lenis   | Narrative overview, rule lab, simulations, event console                       |
| Testing         | pytest, Vitest, Playwright                | Unit, integration, end-to-end and browser checks                               |

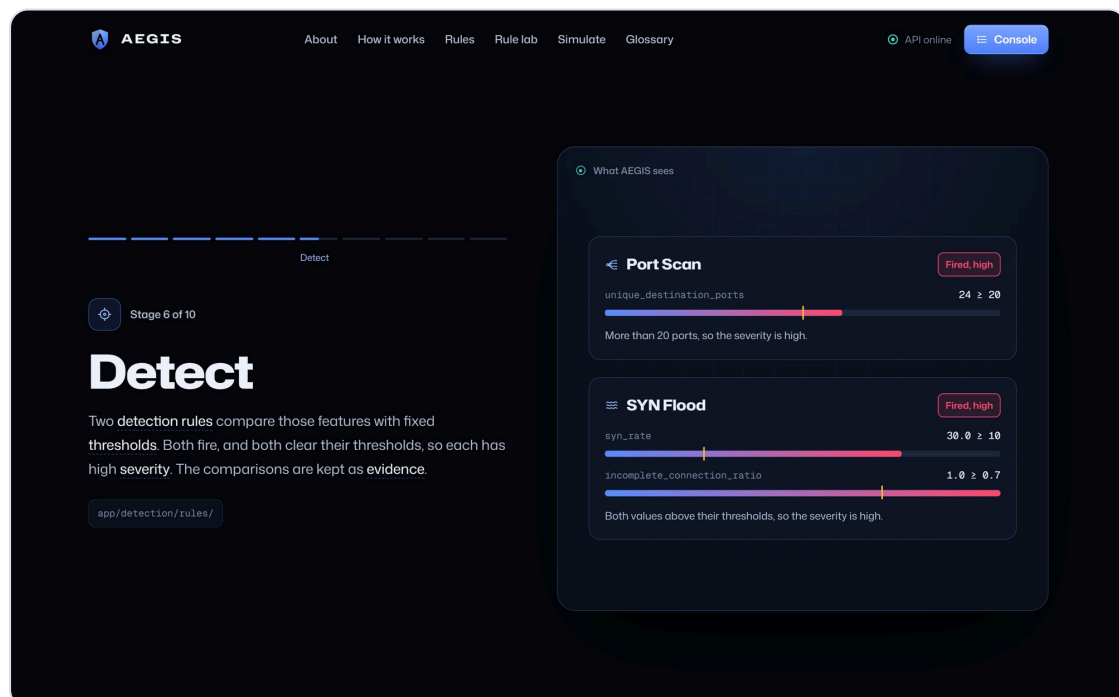
- **Pure components.** Rules, scorer, policy and response engines are stateless and side-effect free, so each is tested on its own.
- **Fault isolation.** A failure in any stage drops only the observation being processed; capture keeps running.
- **Honest simulations.** Simulated traffic uses RFC 5737 documentation addresses, which never occur on real networks, so the interface tags those events as synthetic.

## 5 Web interface

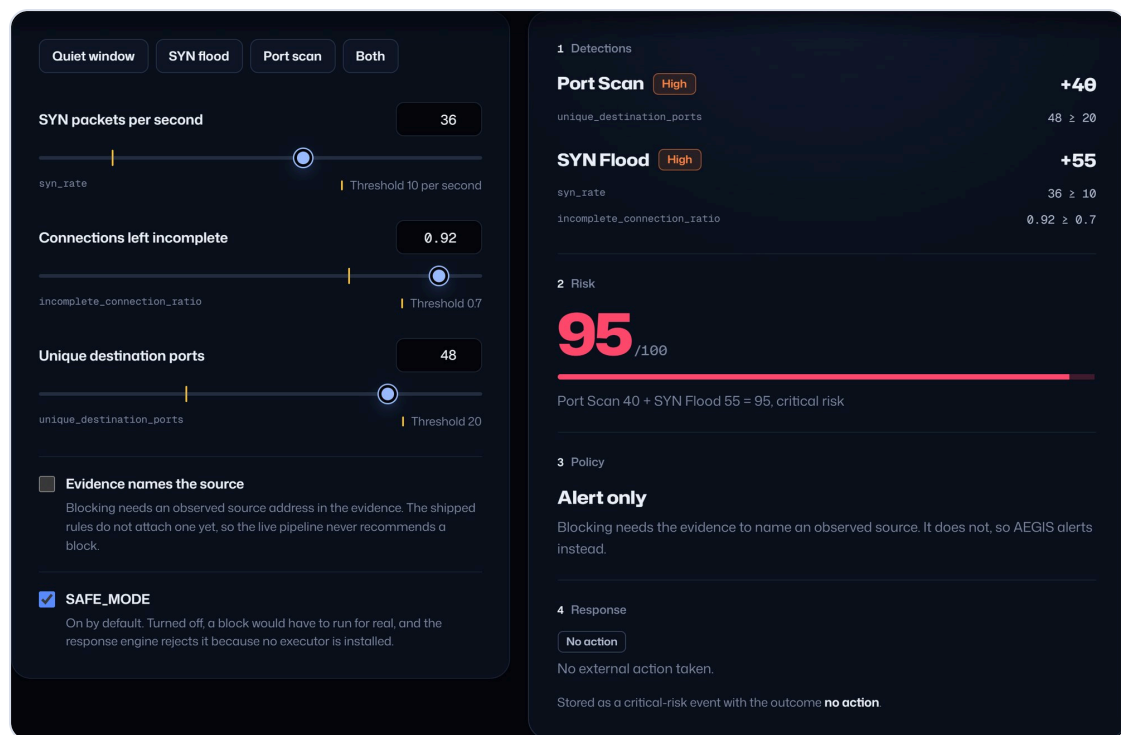
The interface explains AEGIS step by step and shows only real data: every score on screen comes from the API or from the engine's own rules.



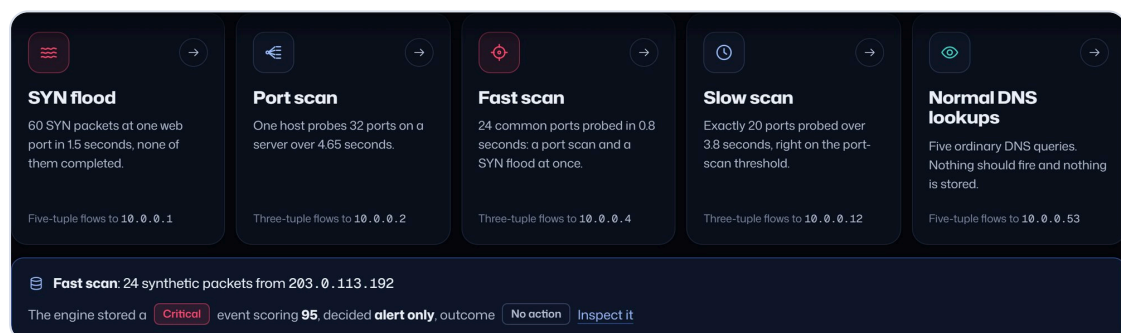
**Figure 3.** Overview. The opening screen states what AEGIS does; the instrument and packet stream are illustrative.



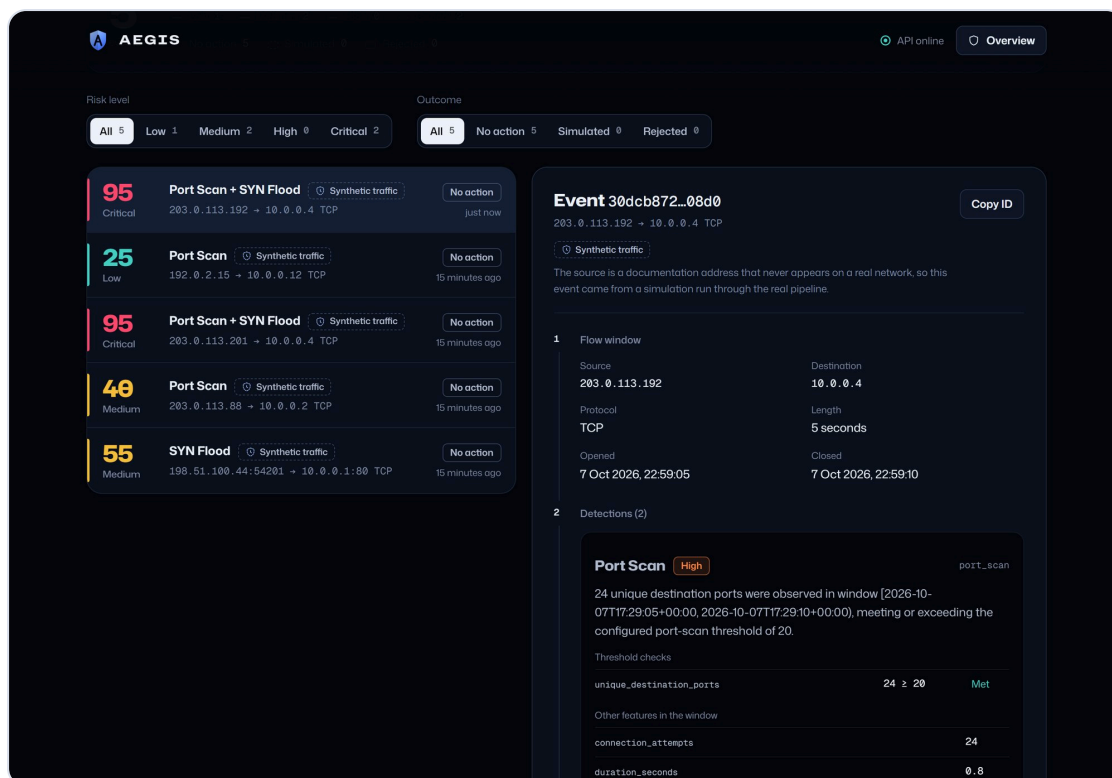
**Figure 4.** "Follow one attack through AEGIS". Scrolling walks through all ten stages; here both rules fire on a fast scan of 24 ports. Underlined terms open a definition.



**Figure 5.** Rule lab. Feature values set by the reader run through a browser copy of the rules, scorer and policy, which is tested against the Python engine's numbers.



**Figure 6.** Attack simulator. Each card asks the API to run synthetic packets through the real pipeline; the result shows what the engine stored.



**Figure 7.** Event ledger. Each stored event opens into its full chain: flow window, detections with threshold checks, risk, policy and response.

The interface is keyboard accessible, keeps filters and the selected event in the URL, and switches off all motion for readers who prefer reduced motion. Only transform and opacity animate.

## 6 Testing and results

307 of 310 automated tests pass. The three failures are one known configuration mismatch, explained below.

| Suite                                                                                                                                                      | Tests | Result               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|----------------------|
| Backend (pytest, 12 files): parser, flows, features, rules, scoring, policy, response<br>safety, events, pipeline, API, simulations, end-to-end validation | 280   | 277 passed, 3 failed |
| Web interface (Vitest, 3 files): rule engine against Python's numbers, API client, pages                                                                   | 30    | 30 passed            |

The three failing tests expect the API to allow only local origins, while the API allows any origin so that the hosted interface can reach it. Reconciling the two is listed as future work.

## Engine output for each simulated attack

| Scenario   | Traffic                 | Rules fired                        | Score | Level    | Decision   |
|------------|-------------------------|------------------------------------|-------|----------|------------|
| Slow scan  | 20 ports in 3.8 s       | Port Scan, medium                  | 25    | Low      | Log only   |
| Port scan  | 32 ports in 4.65 s      | Port Scan, high                    | 40    | Medium   | Alert only |
| SYN flood  | 60 SYN packets in 1.5 s | SYN Flood, high                    | 55    | Medium   | Alert only |
| Fast scan  | 24 ports in 0.8 s       | Port Scan and SYN Flood, both high | 95    | Critical | Alert only |
| Normal DNS | 5 queries               | None                               | -     | -        | Not stored |

Every scenario produced the expected verdict, and benign traffic produced none. The critical fast scan is not blocked, because no rule yet records an observed source; the policy correctly refuses to guess.

## Interface performance

Measured on the production build with the CPU slowed fourfold: scrolling the whole overview gives a median frame time of 16.7 ms (60 frames per second), a 95th percentile of 50 ms, and 3 of 819 frames over 100 ms.

## Running AEGIS

| Task                                                  | Command                                                                    |
|-------------------------------------------------------|----------------------------------------------------------------------------|
| Install and test the backend                          | <code>pip install -r requirements.txt, then python -m pytest tests/</code> |
| Seed demo events from the real engine                 | <code>python -m scripts.seed_demo_attacks</code>                           |
| Start the API                                         | <code>uvicorn app.api.main:app --port 8000</code>                          |
| Start the web interface                               | <code>cd dashboard, npm install, npm run dev</code>                        |
| Capture live traffic (admin rights, Npcap or libpcap) | <code>python -m app.main</code>                                            |

## 7 Limitations and future work

### Current limitations

- Two attack types only: SYN floods and port scans.
- The default five-tuple flow key splits a scan into one flow per port, so port scans need the three-tuple key.
- Rules do not yet record an observed source, so live traffic never leads to a block, and all responses are simulated.
- Alerts arrive late: a window is reported about 50 seconds after it ends under steady traffic, later when traffic is sparse.
- Completed connections are approximated by FIN packets, so short normal connections can look incomplete.
- The hosted demo serves stored and simulated events; it does not capture traffic.

### Future work

- Attach an observed source to detections, then add a guarded firewall executor behind `SAFE_MODE` and human approval.
- Make the flow key a setting and run the port-scan rule on three-tuple flows alongside five-tuple ones.
- Close idle windows on a timer and track TCP connection state for faster, more accurate alerts.
- Add rules for ICMP floods, RST anomalies and brute force; later, an anomaly model whose outputs still carry evidence.
- Live updates over WebSocket, authentication for simulations, and an origin allowlist for the API.

## 8 Conclusion

AEGIS shows that intrusion detection can be transparent without being trivial. Every alert it raises carries the measurements, thresholds and decisions behind it, and the same traffic always produces the same verifiable record. The response layer is safe by design, and the web interface makes each step understandable to someone new to the field.

Its limits are documented and measurable, which makes the next steps clear: wider attack coverage, faster alerts, and a carefully guarded path from simulated to real response.

### References

1. W. Eddy, Ed., *Transmission Control Protocol (TCP)*, RFC 9293, IETF, 2022.
2. W. Eddy, *TCP SYN Flooding Attacks and Common Mitigations*, RFC 4987, IETF, 2007.
3. J. Arkko, M. Cotton, L. Vegoda, *IPv4 Address Blocks Reserved for Documentation*, RFC 5737, IETF, 2010.
4. S. McCanne, V. Jacobson, "The BSD Packet Filter: A New Architecture for User-level Packet Capture", USENIX Winter Conference, 1993.
5. M. Roesch, "Snort: Lightweight Intrusion Detection for Networks", USENIX LISA, 1999.
6. Scapy documentation, [scapy.net](https://scapy.net); FastAPI documentation, [fastapi.tiangolo.com](https://fastapi.tiangolo.com).